

윗놀이 강화학습 프로젝트 제안서

강화학습 수업 - 팀 프로젝트 (2인) 마감일: 2026년 6월 12일 (금) 작성일: 2026-05-07

1. 프로젝트 개요

1.1 한 줄 요약

두 팀원이 서로 다른 강화학습 알고리즘으로 윗놀이 AI를 학습시키고, 동일한 환경에서 비교 실험을 통해 알고리즘별 성능과 학습된 정책의 특성을 분석한다.

1.2 왜 윗놀이인가?

- 한국 전통 게임 - 학술적 RL 연구가 거의 없음, 차별화 가능
- 확률과 전략의 결합 - 윗의 무작위성과 말 운용 전략이 RL에 적합
- 적절한 복잡도 - 바둑처럼 거대하지 않으나 단순 게임보다 풍부
- 선행 사례 존재 - NHN FORWARD 2022 발표로 가능성 검증됨
- 명확한 의사결정 포인트 - 풀(pool) 순서, 잡기, 업기, 분기점 활용

1.3 프로젝트의 차별화 포인트

- 여러 RL 알고리즘 비교 - 단일 알고리즘이 아닌 다각도 접근
- 풀(pool) 순서 결정 학습 - 한 턴 내 여러 윗 결과의 적용 순서 최적화
- 정책 해석 분석 - RL이 잡기/업기/지름길을 언제 선택하는지 정성 분석
- Reward 설계 영향 분석 - Sparse vs Shaped reward 비교

2. 윗놀이 룰 명세서 v1.0 (확정)

위키백과(<https://ko.wikipedia.org/wiki/윗놀이>) 기반으로 확정

2.1 게임 기본

항목	규칙
플레이어 수	2명
말 개수	4개/플레이어
승리 조건	4개 말 모두 출발점(참먹이) 통과

2.2 말의 상태

- WAITING: 대기 (아직 보드 진입 안 함)
- ON_BOARD: 보드 위 이동 중
- FINISHED: 출발점 통과 완료

2.3 윷 결과 및 보너스

결과	이동	자동 보너스
뒷도	-1	없음
도	+1	없음
개	+2	없음
걸	+3	없음
윷	+4	있음
모	+5	있음

2.4 보너스 턴 규칙 (핵심)

- 자동 보너스: 윷, 모가 나오면 1회 추가 던지기
- 잡기 보너스: 도/개/걸/뒷도로 잡으면 1회 추가 던지기
- 윷/모로 잡기: 보너스 중복 안 됨 (이미 자동 보너스 있음)
- 던진 결과들은 풀(pool)에 누적되며, 순서를 자유롭게 적용 가능

2.5 말길 (4가지 경로)

- 윷판에는 4가지 가능한 경로가 존재
- 모서리/중앙에 정확히 멈추면 사선(지름길) 진입
- 통과만 하면 외곽으로 진행
- 분기는 자동 결정 (선택권 없음)

2.6 업기 / 잡기

- 자기 말이 같은 칸 도착 시 자동 업기, 분리 불가
- 도착 칸에 상대 말 있으면 잡기, 상대 말은 대기 공간으로
- 업힌 말은 통째로 잡힘
- 거쳐가는 칸은 잡기 불가, 도착 칸만 판정

2.7 골인 (남)

- 출발점을 통과해야 골인
- 정확히 도착 시 머무름 (즉시 골인 아님)
- 머문 상태에서 다음 양수 결과로 즉시 골인
- 출발점에 멈춰있는 상대 말은 잡기 가능

2.8 뒷도 및 기타 규칙 (확정)

항목	결정
대기 중인 말의 뒷도 사용	불가
후진 후 즉시 남	불가
말 없을 때 뒷도 (날뫼 진입)	미사용
낙(落) 처리	미사용
함정(풍당)	추가 안 함

3. 강화학습 설계

3.1 State (상태) 설계

보드 노드 구성 (총 약 31개)

- 외곽 노드: 20개 (모서리 4 + 일반 16)
- 지름길 노드: 8개
- 중앙 노드: 1개 (또는 진입 방향에 따라 분할)
- 가상 노드: WAITING, FINISHED

두 가지 표현

표현 A: Tabular (Q-Learning용)

```
state_tuple = (  
    sorted(tuple(my_pieces_positions)),  
    sorted(tuple(opp_pieces_positions)),  
    pool_count_tuple, # (뒷도, 도, 개, 걸, 뫼, 모) 각 개수  
)
```

표현 B: Vector (DNN용)

```
state_vector = concat([  
    내_말_위치_one_hot,           # 4 × 31  
    내_말_상태_one_hot,          # 4 × 3  
    내_업힘_정보,                 # 4 × 4  
    상대_말_위치_one_hot,        # 4 × 31  
    상대_말_상태_one_hot,        # 4 × 3  
    상대_업힘_정보,              # 4 × 4  
    풀_카운트,                   # 6  
)  
# 총 약 310차원 (압축 가능)
```

3.2 Action (행동) 설계

핵심 결정

풀(pool)의 어느 윗 결과를, 어느 말에 적용할 것인가

Action Space (고정 크기)

```
action = (piece_id, yut_type)
# piece_id: 0~3 (내 말 4개)
# yut_type: 0~5 (뒷도/도/개/걸/윗/모)
# Action space 크기:  $4 \times 6 = 24$ 
```

Action Masking (필수)

불법 액션 케이스:

- 풀에 해당 yut_type이 없음
- 말이 FINISHED 상태
- 말이 WAITING 상태이면서 yut_type이 뒷도

3.3 Reward (보상) 설계

옵션 A: Sparse Reward

```
R = +100 if 승리
R = -100 if 패배
R = 0    otherwise
```

옵션 B: Shaped Reward

```
R = +1.0 x 말 골인
+ 0.5 x 상대 말 잡기
+ 0.1 x 말 진행 거리
- 0.5 x 내 말 잡힘
+ 0.3 x 모서리 정확 도착
+ 100 x 게임 승리
- 100 x 게임 패배
```

결정

둘 다 실험하여 비교 - Reward 설계가 학습된 정책에 미치는 영향 분석

4. 알고리즘 분담안

4.1 추천 조합 (회의에서 선택)

조합 1: 입문자 친화

- 팀원 A: Tabular Q-Learning

- 팀원 B: DQN
- 분석 포인트: "신경망이 정말 필요한가?"

조합 2: Value vs Policy

- 팀원 A: DQN (Value-based)
- 팀원 B: REINFORCE 또는 PPO (Policy-based)
- 분석 포인트: "두 패러다임의 장단점"

조합 3: 도전적

- 팀원 A: DQN
- 팀원 B: AlphaZero-lite (MCTS + NN)
- 분석 포인트: "탐색 + 학습의 시너지"
- 주의: 시간 부족 위험

4.2 추천: 조합 1 또는 2

- 시간 안에 안전하게 완성 가능
- 두 알고리즘 모두 입문자도 구현 가능한 수준
- 비교 결과가 명확하게 나옴

5. R&R 분담

5.1 공동 작업 (1단계: 환경 구축)

- 룰 명세서 검토 및 합의
- 보드 그래프 설계
- 게임 로직 / 윗 시뮬레이션 / 풀 처리
- State / Action / Reward 인코딩 합의
- Agent 인터페이스 정의
- 베이스라인 구현 (Random, Heuristic)
- 평가 시스템 (Tournament, 메트릭)

5.2 개별 작업 (2단계: 알고리즘 학습)

영역	팀원 A	팀원 B
환경 - 보드 그래프	담당	
환경 - 게임 로직/윗		담당
베이스라인	Random, Greedy	Aggressive, Defensive
RL 알고리즘	(조합에 따라)	(조합에 따라)
하이퍼파라미터 튜닝	A의 알고리즘	B의 알고리즘
정책 분석	A의 정책	B의 정책

5.3 공동 작업 (3단계: 비교 및 보고서)

- 두 알고리즘 직접 대전 (Tournament)

- 비교 분석 (정량 + 정성)
 - Reward 설계 영향 실험
 - 보고서 작성 (각자 담당 부분 + 통합)
 - 발표 자료 준비
 - GitHub README 작성
-

6. 실험 설계

6.1 실험 목록

실험 1: 알고리즘별 성능 비교 (메인)

- A 알고리즘 vs Baseline 4종
- B 알고리즘 vs Baseline 4종
- A vs B 직접 대전

실험 2: Reward 설계 영향

- Sparse vs Shaped (각 알고리즘에서)
- 학습 곡선 비교
- 학습된 정책의 차이

실험 3: 학습된 정책의 정성 분석

- 잡기 패턴: 어떤 옷으로 잡는가? (도/개/걸 우선?)
- 분기점 활용: 모서리 정확 도착 비율
- 업기 빈도: 위험 감수 정도
- 풀 순서 결정: 동일 상황에서 알고리즘별 차이

실험 4: Seed 변경 신뢰구간 (평가 기준 필수)

- 각 알고리즘 5개 seed로 학습
- 평균과 표준편차 그래프

실험 5 (선택): 룰 변형

- 말 개수 변경 (2개/3개/4개)
- 이론 확률 vs 측정 확률 비교

6.2 평가 메트릭

정량 메트릭

메트릭	측정 방법
승률 (vs Random)	1000판 평균
승률 (vs Greedy)	1000판 평균
승률 (vs Heuristic)	1000판 평균
승률 (A vs B)	직접 대전
학습 속도	에피소드별 reward 곡선
샘플 효율성	50% 승률 도달까지 episode
수렴 안정성	seed별 분산
평균 게임 길이	의사결정 효율성

정성 메트릭

메트릭	분석 방법
잡기 패턴	윗 결과별 잡기 빈도 분포
분기점 활용	모서리/중앙 정확 도착 비율
업기 패턴	업기 발생 빈도
폴 순서	동일 상태에서 선택 차이

7. 일정 계획

7.1 전체 일정

주차	팀원 A	팀원 B	마일스톤
1-2	보드 그래프 구현	윗/폴/마스킹 구현	환경 v1.0
3	Random, Greedy	Aggressive, Defensive	베이스라인 4종
4-5	A 알고리즘 학습	B 알고리즘 학습 시작	첫 학습 결과
6-7	A 알고리즘 튜닝	B 알고리즘 완료	두 알고리즘 완성
8	추가 실험 (Sparse vs Shaped)	추가 실험	
9	A vs B Tournament	A vs B Tournament	비교 결과
10	추가 실험 (롤 변형)	추가 실험	
11	정책 분석 (A)	정책 분석 (B)	정성 분석
12	보고서 (담당 부분)	보고서 (담당 부분)	초안
13	통합, README	통합, 시각화	최종 리뷰
14	마감 정리	마감 정리	제출 (6/12)

8. 보고서 구조 (PPT)

8.1 슬라이드 목차

1. 표지 - 팀원 학번/이름, GitHub 링크
 2. 프로젝트 개요 및 목표
 3. 윗놀이 룰 명세 - 확정 룰 정리
 4. 환경 설계 - 보드 그래프, 시각화
 5. State / Action / Reward 설계
 6. 베이스라인 소개 - Random, Heuristic 4종
 7. [팀원 A] 알고리즘 X 설명 + 학습 결과
 8. [팀원 B] 알고리즘 Y 설명 + 학습 결과
 9. 비교 실험: 정량 결과 - 승률 표, 학습 곡선
 10. 비교 실험: 정성 분석 - 정책 차이 분석
 11. Reward 설계 영향 분석
 12. 토의 및 결론
 13. 한계와 향후 과제
 14. 기여도 명시 (필수)
-

9. 코드 구조 제안

```
yutnori-rl/
├── README.md           # GitHub 메인 페이지
├── requirements.txt     # 의존성
├── env/
│   ├── __init__.py
│   ├── board.py        # 보드 그래프 (팀원 A)
│   ├── yut.py          # 윷 던지기 (팀원 B)
│   ├── game.py         # 게임 룰 통합
│   └── yutnori_env.py   # Gym-style wrapper
├── agents/
│   ├── __init__.py
│   ├── base.py         # Agent 추상 클래스
│   ├── random_agent.py # 베이스라인
│   ├── heuristic_agent.py # 베이스라인
│   ├── algo_a.py       # 팀원 A 알고리즘
│   └── algo_b.py       # 팀원 B 알고리즘
├── train/
│   ├── train_a.py      # A 학습 스크립트
│   └── train_b.py      # B 학습 스크립트
├── evaluate/
│   ├── tournament.py   # agent 간 대전
│   └── analysis.py     # 성능 분석
├── visualize/
│   ├── render.py       # 게임 시각화
│   └── plots.py        # 그래프
├── configs/            # 하이퍼파라미터
├── results/            # 학습 결과 저장
├── notebooks/         # 분석 노트북
├── docs/
│   ├── PROJECT_PROPOSAL.md # 이 문서
│   ├── RULES.md          # 룰 명세서
│   └── INTERFACE.md      # API 명세
```

10. 회의 안건 (오늘 회의)

10.1 결정 사항

- [] 프로젝트 주제 최종 합의 (윷놀이 RL)
- [] 룰 명세서 v1.0 검토 및 확정
- [] 알고리즘 조합 선택 (조합 1 / 2 / 3)
- [] 누가 어떤 알고리즘 담당 (선호 + 강점 기반)
- [] State 표현 차원 합의
- [] Reward 설계 (Sparse / Shaped / 둘 다)
- [] 환경 코드 분담 확정
- [] Git workflow (브랜치, PR 정책)
- [] 다음 회의 일정

10.2 합의 인터페이스

```
# Environment API
class YutnoriEnv:
    def reset() -> state
    def step(action) -> (state, reward, done, info)
    def get_legal_actions() -> List[action]
    def render()

# Agent Interface
class BaseAgent:
    def select_action(state, legal_actions) -> action
    def update(state, action, reward, next_state, done)
    def save(path)
    def load(path)
```

10.3 다음 액션 아이템 (회의 후)

- 팀원 A: _____
- 팀원 B: _____
- 다음 회의: _____

11. 평가 기준 매핑

수업 평가 기준에 본 프로젝트가 어떻게 부합하는지:

평가 항목	점수	본 프로젝트의 강점
문제 정의 및 환경 구현	25	윷놀이 룰 명세 정확, 보드 그래프 정밀 구현
알고리즘 구현 및 실험 설계	30	2개 RL 알고리즘 + 4개 baseline + 다각도 실험
결과 분석 및 해석	30	정량 + 정성 분석, seed별 신뢰구간, 정책 비교
보고서 및 코드 완성도	15	명확한 코드 구조, GitHub README, 기여도 명시

12. 참고 자료

12.1 선행 연구

- NHN FORWARD 2022: "강화학습으로 사람보다 잘하는 윷놀이 AI 만들기"
 - <https://forward.nhn.com/2022/sessions/25>
 - YouTube: <https://www.youtube.com/watch?v=ATvSMkJwCbA>
- 퍼지논리를 이용한 윷놀이 인공지능 플레이어 연구 (2017, 정성욱·김기년)
 - 디지털산업정보학회 논문지

12.2 를 참고

- 위키백과 윷놀이: <https://ko.wikipedia.org/wiki/윷놀이>

12.3 RL 학습 자료

- Sutton & Barto, "Reinforcement Learning: An Introduction"
 - OpenAI Spinning Up: <https://spinningup.openai.com>
 - Gymnasium: <https://gymnasium.farama.org>
-

부록: 향후 확장 가능 주제

시간이 남으면 추가 실험 가능:

- 4인 윗놀이 (멀티 에이전트)
- 실제 윗 확률 분포로 학습 vs 이론 확률
- 룰 변형 비교 (말 개수, 보드 크기)
- Self-play vs Frozen opponent 학습 방식 비교
- 인간 플레이어 대상 평가